Faculty of Computer Science and Information Technology

CCS3300-1 : OPERATING SYSTEMS

SEMESTER 1 2025/2026

PROJECT REPORT
Memory Placement Algorithm and Frame Replacement Algorithm

LECTURER NAME :
NUR IZURA BINTI UDZIR

PROVIDED BY :

NUM.	NAME	MATRIC No.
1.	YUE CHENGHAO	227154
2.	HUA JIE	226758
3.	WANG KAILUN	227046

Project Report Appendix

1.0 INTRODUCTION

Effective memory management is a critical function of modern Operating Systems (OS), ensuring that processes access system resources efficiently. The main objective of this project is to simulate and analyze the behavior of core memory management algorithms using the Java programming language.

This project is divided into two main parts:

1. **Memory Placement Algorithms:** We simulate how the OS allocates memory partitions to incoming processes. specifically implementing the **First Fit**, **Next Fit**, and **Best Fit** algorithms. This section aims to observe how different strategies affect memory allocation and fragmentation .
2. **Frame Replacement Algorithms:** We simulate Virtual Memory management by implementing **First-In-First-Out (FIFO)**, **Least Recently Used (LRU)**, and the **Optimal** algorithm. The goal is to compare these strategies based on the number of **Page Faults** they generate under limited physical memory constraints .

By analyzing the output of these simulations, this report evaluates the efficiency, trade-offs, and practical performance of each algorithm in managing system resources .

2.0 PART 1: MEMORY PLACEMENT ALGORITHM

2.1 Introduction & Objective

The primary objective of this section is to simulate and understand the memory placement algorithms commonly used in operating systems. In a dynamic memory environment, the operating system must decide which free memory partition to allocate to a process. This project implements three fundamental algorithms using the Java programming language:

- **First Fit:** Allocates the first available partition that is large enough.
- **Next Fit:** Starts the search from the location of the last allocation.
- **Best Fit:** Searches the entire list to find the smallest partition that fits the process.

By implementing these algorithms, we aim to analyze their behavior regarding memory utilization and fragmentation management .These algorithms represent different trade-offs between allocation speed, memory utilization, and fragmentation, which are key concerns in operating system memory management.

2.2 Problem Scenario

The simulation is based on a specific initial state of the memory. The system contains a list of memory partitions with defined sizes (measured in Kilobytes/Kb). Some partitions are already occupied by existing segments (marked with 'X'), while one partition is marked as the most recent placement (marked with 'R').

Initial Memory Layout: The specific memory partitions provided for this simulation are as follows :

- **Partition 0:** 100 Kb (Free)
- **Partition 1:** 20 Kb (Occupied - 'X')
- **Partition 2:** 80 Kb (Free)
- **Partition 3:** 50 Kb (Free, marked 'R' as Recent Placement)
- **Partition 4:** 50 Kb (Free)
- **Partition 5:** 120 Kb (Occupied - 'X')
- **Partition 6:** 100 Kb (Free)

This predefined layout is designed to highlight how different placement strategies behave under fragmentation and constrained memory conditions.

2.3 Code Implementation & Explanation

The solution is implemented in Java, utilizing an object-oriented approach to represent memory partitions and a main driver class to execute the algorithms.

```
import java.util.*;

/**
* PROJECT: Memory Placement Algorithm
* * This program simulates three memory management algorithms:
* 1. First Fit
* 2. Next Fit
* 3. Best Fit
* * It takes a list of memory partition sizes and a list of new process requests
* as input, and outputs the final state of the memory partitions.
*/
public class MemoryPlacement {

  /**
```

```
 * Represents a single memory partition.
 * Keeps track of its total size, current usage, and status (Occupied/Recent).
 */
static class Partition {
    int totalSize;              // Total capacity of the partition in Kb
    int currentSize;            // Currently used space in Kb
    boolean isOccupied;         // True if pre-occupied by system (marked 'X')
    String label;               // Label for initial state (e.g., "R" for Recent)
    List<Integer> allocations;  // List of process sizes allocated to this partition

    /**
     * Constructor to initialize a partition.
     * * @param size       The total size of the partition.
     * @param isOccupied Whether the partition is blocked ('X').
     * @param label      Special label like "R" (Recent), or null.
     */
    public Partition(int size, boolean isOccupied, String label) {
        this.totalSize = size;
        this.currentSize = 0;
        this.isOccupied = isOccupied;
        this.label = label;
        this.allocations = new ArrayList<>();
    }

    /**
     * Copy Constructor.
     * Used to reset the memory state between different algorithm runs
     * so they don't interfere with each other.
     */
    public Partition(Partition other) {
        this.totalSize = other.totalSize;
        this.currentSize = other.currentSize;
        this.isOccupied = other.isOccupied;
        this.label = other.label;
        this.allocations = new ArrayList<>(other.allocations);
    }
```

```java
/**
 * Calculates the remaining free space in this partition.
 * @return Available space in Kb. Returns 0 if occupied by 'X'.
 */
public int getFreeSpace() {
    if (isOccupied) return 0;
    return totalSize - currentSize;
}

/**
 * Tries to allocate memory for a process of specific size.
 * * @param size The size of the process to place.
 * @return true if allocation was successful, false otherwise.
 */
public boolean allocate(int size) {
    if (isOccupied) return false;

    // Check if there is enough space
    if (getFreeSpace() >= size) {
        allocations.add(size);
        currentSize += size;
        return true;
    }
    return false;
}

/**
 * Generates the string representation for output.
 * Format examples: "X", "R", "-", "20", "20, 30"
 */
@Override
public String toString() {
    // 1. If it is a system occupied partition, print "X"
    if (isOccupied) return "X";
```

```
        // 2. If no new pages have been placed here yet
        if (allocations.isEmpty()) {
            // If it has a label (like "R"), print it, otherwise print "-"
            return (label != null) ? label : "-";
        }

        // 3. If pages are placed, list them (e.g., "40, 30")
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < allocations.size(); i++) {
            sb.append(allocations.get(i));
            if (i < allocations.size() - 1) sb.append(", ");
        }
        return sb.toString();
    }
}

/**
 * Helper method to initialize the specific scenario described in the Project PDF.
 * Layout: 100, 20(X), 80, 50(R), 50, 120(X), 100
 */
private static List<Partition> getInitialPartitions() {
    List<Partition> list = new ArrayList<>();
    list.add(new Partition(100, false, null));
    list.add(new Partition(20, true, "X"));
    list.add(new Partition(80, false, null));
    list.add(new Partition(50, false, "R")); // Index 3 is the Recent placement
    list.add(new Partition(50, false, null));
    list.add(new Partition(120, true, "X"));
    list.add(new Partition(100, false, null));
    return list;
}

/**
 * Helper method to print the list of partitions in the required format.
 * Example output: "100, X, 20, R, 20, X, 40, 30"
 */
```

```
private static void printPartitions(List<Partition> partitions) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < partitions.size(); i++) {
        sb.append(partitions.get(i).toString());
        // Add a comma separator after each partition, except the last one
        if (i < partitions.size() - 1) {
            sb.append(", ");
        }
    }
    System.out.print(sb.toString());
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // --- Step 1: Input Handling ---
    System.out.print("Specify total insertion(s) : ");
    int n = 0;
    try {
        n = scanner.nextInt();
    } catch (Exception e) {
        System.out.println("Invalid input. Please enter an integer.");
        return;
    }

    System.out.print("Input " + n + " insertion(s) in Kb: ");
    int[] requests = new int[n];
    for (int i = 0; i < n; i++) {
        requests[i] = scanner.nextInt();
    }

    System.out.println("Result:");

    // --- Step 2: Display Initial Scenario ---
    System.out.print("Scenario (R=recent, X=occupied) : ");
    printPartitions(getInitialPartitions());
```

```
        System.out.println();

        // --- Step 3: Execute Algorithms ---
        runFirstFit(requests);
        runNextFit(requests);
        runBestFit(requests);

        scanner.close();
    }

    /**
     * Algorithm 1: First Fit
     * Scans from the beginning of memory (index 0) for the first hole that is large enough.
     */
    private static void runFirstFit(int[] requests) {
        // Load fresh partition data
        List<Partition> partitions = getInitialPartitions();
        boolean success = true;

        for (int req : requests) {
            boolean placed = false;

            // Always start scanning from the first partition
            for (Partition p : partitions) {
                if (p.allocate(req)) {
                    placed = true;
                    break; // Stop scanning once placed
                }
            }

            // Error handling if no partition matches
            if (!placed) {
                System.out.println("First-fit placement : Error: the remaining memory is not
enough to place more pages");
                success = false;
                break;
```

```
            }
        }

        if (success) {
            System.out.print("First-fit placement : ");
            printPartitions(partitions);
            System.out.println();
        }
    }

    /**
     * Algorithm 2: Next Fit
     * Begins scanning from the location of the last placement (pointer).
     * Based on PDF: 'R' indicates the last placement. Start scanning AFTER 'R'.
     */
    private static void runNextFit(int[] requests) {
        List<Partition> partitions = getInitialPartitions();
        boolean success = true;

        // 1. Locate the 'R' partition to initialize the pointer
        int lastAllocatedIndex = 0;
        for(int i = 0; i < partitions.size(); i++) {
            if ("R".equals(partitions.get(i).label)) {
                lastAllocatedIndex = i;
                break;
            }
        }

        // The pointer starts at the partition immediately FOLLOWING 'R'
        int currentPointer = (lastAllocatedIndex + 1) % partitions.size();

        for (int req : requests) {
            boolean placed = false;
            int partitionsChecked = 0;

            // Loop through partitions, but limit to one full circle (size of list)
```

```
        while (partitionsChecked < partitions.size()) {
            Partition p = partitions.get(currentPointer);

            // Try to allocate in the current pointer location
            if (p.allocate(req)) {
                placed = true;
                // Note: In Next Fit, if allocation is successful,
                // the pointer STAYS here to use the remaining space for the next request.
                // We break the inner loop to process the next request.
                break;
            }

            // If not successful, move pointer to the next partition (wrap around using modulo)
            currentPointer = (currentPointer + 1) % partitions.size();
            partitionsChecked++;
        }

        if (!placed) {
            System.out.println("Next-fit placement : Error: the remaining memory is not
enough to place more pages");
            success = false;
            break;
        }
    }

    if (success) {
        System.out.print("Next-fit placement : ");
        printPartitions(partitions);
        System.out.println();
    }
}

/**
 * Algorithm 3: Best Fit
 * Scans the entire list and allocates to the smallest hole that is big enough.
 */
```

```
private static void runBestFit(int[] requests) {
    List<Partition> partitions = getInitialPartitions();
    boolean success = true;

    for (int req : requests) {
        int bestIndex = -1;
        int minFrag = Integer.MAX_VALUE; // Start with max possible fragmentation

        // Scan all partitions to find the "Tightest" fit
        for (int i = 0; i < partitions.size(); i++) {
            Partition p = partitions.get(i);

            // Partition must be free (not X) and have enough space
            if (!p.isOccupied && p.getFreeSpace() >= req) {
                int remainingSpace = p.getFreeSpace() - req;

                // If this partition offers a tighter fit than the previous best, update bestIndex
                if (remainingSpace < minFrag) {
                    minFrag = remainingSpace;
                    bestIndex = i;
                }
            }
        }

        // If a suitable partition was found, allocate it
        if (bestIndex != -1) {
            partitions.get(bestIndex).allocate(req);
        } else {
            System.out.println("Best-fit placement : Error: the remaining memory is not
enough to place more pages");
            success = false;
            break;
        }
    }

    if (success) {
```

```
        System.out.print("Best-fit placement : ");
        printPartitions(partitions);
        System.out.println();
    }
  }
}
```

A. Data Structure: The Partition Class

To manage memory effectively, a Partition class was designed to represent individual memory blocks .The Partition class abstracts a contiguous memory block and encapsulates both its capacity and allocation state, allowing the placement algorithms to operate independently of low-level memory details.

This class encapsulates the following attributes:

- totalSize: The total capacity of the partition.
- currentSize: The amount of memory currently used.
- isOccupied: A boolean flag indicating if the partition is reserved by the system (marked 'X').
- label: A marker for special states, such as "R" for the recent placement.
- allocations: A list to track the size of individual processes placed within the partition.
-

B. Algorithm Logic

The program takes a list of integer requests (process sizes) as input and processes them according to the rules of each algorithm.All three placement algorithms share the same memory representation but differ in their search strategy for selecting a suitable partition.

1. First Fit Algorithm

- **Logic:** For every new process request, the algorithm scans the partition list from the very beginning (Index 0). It places the process in the **first** partition that has sufficient free space (freeSpace >= requestSize) .Due to its greedy nature, First Fit often causes external fragmentation to accumulate near the beginning of memory.
- **Complexity:** The time complexity is **O(N)** in the worst case, as it may scan the entire list to find a spot. However, it is generally fast because it stops as soon as a match is found.

2. Next Fit Algorithm

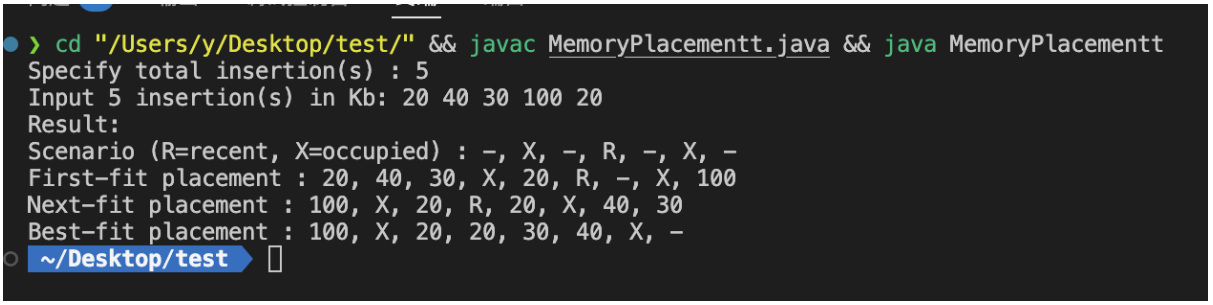
- **Logic:** This algorithm maintains a global pointer indicating where the last allocation occurred. Based on the problem scenario, the pointer is initially set to the partition immediately following the 'R' marker (Index 3). For a new request, the search begins at this pointer and wraps around to the beginning of the list if the end is reached .
- **Advantage:** This technique distributes allocations more evenly across the memory, potentially reducing the accumulation of small fragments at the beginning of memory.

3. Best Fit Algorithm

- **Logic:** For every request, the algorithm scans **all** available partitions. It calculates the remaining space (fragmentation) that would result if the process were placed in each valid partition. It then selects the partition that creates the smallest remaining hole .
- **Complexity:** The time complexity is **O(N)**, but it is typically slower than First Fit because it **must** iterate through the entire list to ensure the "best" candidate is found.
- **Goal:** This strategy aims to minimize wasted space in large partitions, preserving them for potential future large requests.
- Although Best Fit minimizes immediate wasted space, it may generate many small unusable holes over time, potentially increasing external fragmentation.
-

2.4 Output & Screen Capture

To verify the correctness of the implemented algorithms, the program was executed with the input dataset specified in the project requirements.



Test Input:

- **Total insertions:** 5 pages/segments.
- **Page sizes:** 20 Kb, 40 Kb, 30 Kb, 100 Kb, 20 Kb.

Program Output: The following screenshot demonstrates the final state of the memory partitions after processing the requests using First Fit, Next Fit, and Best Fit algorithms.

2.5 Discussion & Analysis

This section analyzes the efficiency of each algorithm based on the output obtained in Section 2.4, and further explores their behavior through additional test scenarios.

2.5.1 Analysis of Main Scenario

Based on the input of 5 pages (20, 40, 30, 100, 20 Kb), we observed distinct behaviors for each algorithm:

- **First Fit:** Under the given input sequence (20, 40, 30, 100, 20 Kb), the First Fit algorithm allocated the first three requests entirely into the first 100 Kb partition. This behavior is caused by its greedy and sequential scanning strategy, which always selects the earliest available partition that can satisfy the request.

  As a result, memory usage becomes highly concentrated at the beginning of the memory layout, leading to uneven utilization and the accumulation of external fragmentation in early partitions. While First Fit is fast due to minimal searching overhead, this example demonstrates its weakness in long-term memory distribution.

- **Next Fit:** The Next Fit algorithm begins searching from the partition immediately following the most recent placement (marked as ‘R’), rather than restarting from the beginning. In this scenario, this strategy allowed Next Fit to utilize memory regions that were ignored by First Fit, including placing the 100 Kb request into the last available large partition.

  By continuing the search from the last allocation point, Next Fit can potentially reduce the concentration of fragmentation at the beginning of memory. However, its effectiveness still depends on the access pattern and does not guarantee optimal memory utilization.

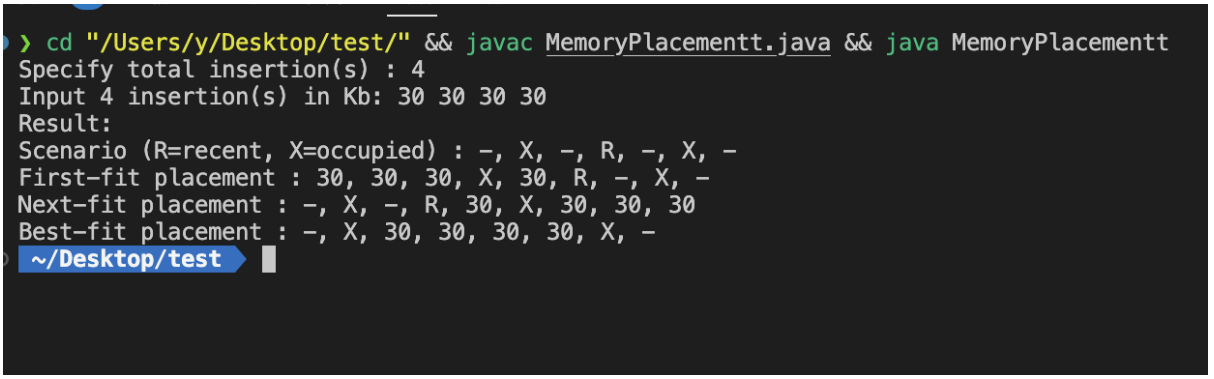
- **Best Fit:** The Best Fit algorithm minimizes immediate wasted space by selecting the partition that results in the smallest remaining free space after allocation. In this experiment, large requests such as 100 Kb were placed into perfectly matching partitions, while smaller requests were allocated to smaller gaps.

  This approach preserves large contiguous memory blocks for future allocations, which is advantageous for handling large processes. However, although Best Fit reduces internal wasted space in the short term, it may generate many small unusable holes over time, potentially increasing external fragmentation.

2.5.2 Sensitivity Analysis (Extended Evaluation)

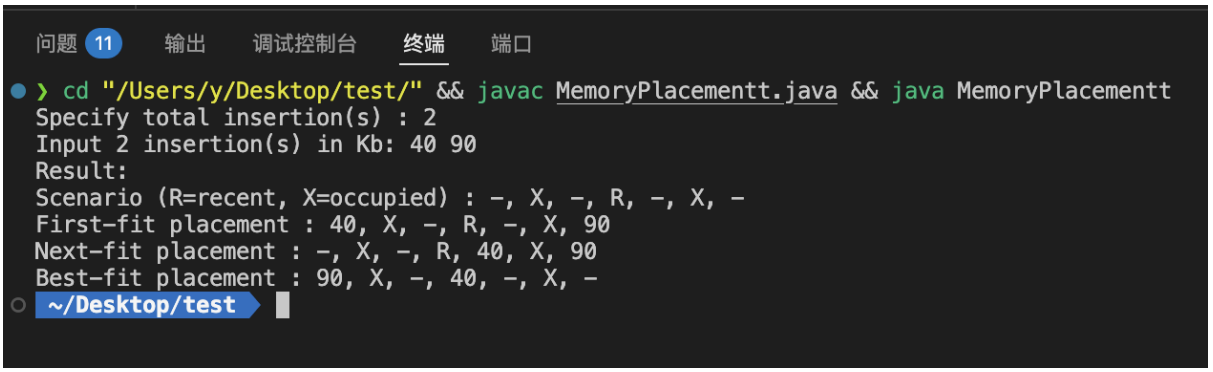
To demonstrate the robustness of the solution and the specific advantages of the Best Fit algorithm, two additional test cases were conducted.

A. Test Case 1: Filling Small Gaps (Input: 30, 30, 30, 30 Kb)



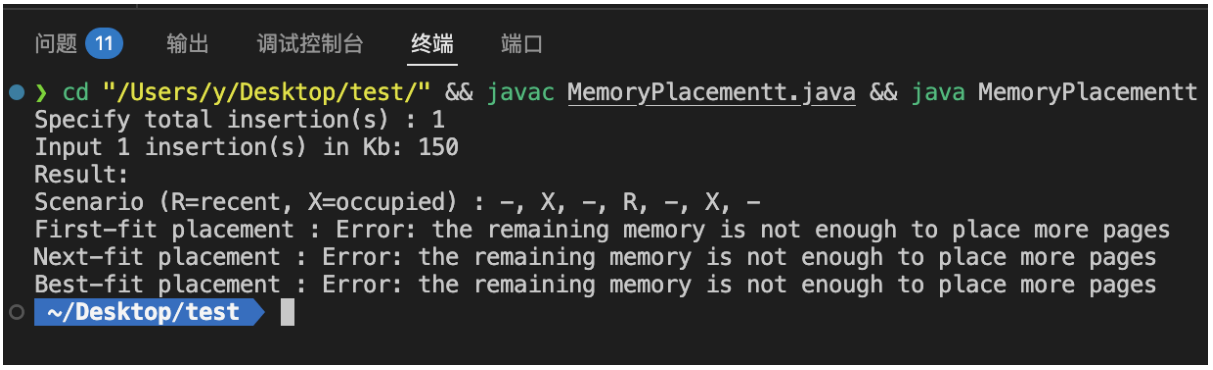
- **Observation:** First Fit stacked the first three 30 Kb requests into the first 100 Kb partition. In contrast, Best Fit distributed these requests into the 50 Kb partitions first.
- **Analysis:** Best Fit proved superior here because it utilized the smaller 50 Kb partitions for the 30 Kb requests, leaving only 20 Kb of remaining space per partition. Crucially, this strategy preserved the larger 100 Kb and 80 Kb blocks for potential future large processes, whereas First Fit immediately fragmented the largest block.

B. Test Case 2: Large Allocation Strategy (Input: 40, 90 Kb) This test highlights the critical weakness of First Fit and the strength of Best Fit.



- **First Fit Behavior:** The algorithm placed the initial 40 Kb request into the first 100 Kb partition. This reduced the available space in that partition to 60 Kb. Consequently, the subsequent 90 Kb request could not fit there and had to be placed in the last partition .
- **Best Fit Behavior:** Best Fit placed the 40 Kb process into a 50 Kb partition (the smallest adequate hole). This decision kept the large 100 Kb partition completely free. If a future request had required the full 100 Kb, Best Fit would have accommodated it, whereas First Fit would have failed to find a contiguous block .

C. Test Case 3: Error Handling



- **Input:** A single request of 150 Kb.
- **Result:** The system correctly identified that the largest available contiguous block is 100 Kb. Since 150 Kb > 100 Kb, all algorithms correctly triggered the required error message: *"Error: the remaining memory is not enough to place more pages"* . This confirms the program's error handling capabilities are robust.

3.0 PART 2: FRAME REPLACEMENT ALGORITHM

3.1 Introduction & Objective

The objective of this section is to simulate the behavior of virtual memory management in an Operating System. Specifically, we aim to implement and compare three frame replacement algorithms: First-In-First-Out (FIFO), Least Recently Used (LRU), and the Optimal algorithm .

In a system with limited physical memory (Frames), the operating system must decide which page to evict (remove) when a new page needs to be loaded. By processing a specific sequence of page demands, we calculate the number of **Page Faults** generated by each algorithm to determine which strategy is most efficient. These algorithms reflect different assumptions about program behavior and locality of reference, which directly influence virtual memory performance.

3.2 Problem Scenario

The simulation is conducted based on the specific parameters provided in the project requirements:

- **Physical Memory Size (F):** 10 Frames.
- **Total Page Demands (N):** 40 Pages.
- **Reference String:** A predefined sequence of 40 integers representing page requests. The sequence begins with unique pages 1 to 20 sequentially, followed by a mix of repeated accesses (e.g., 1, 2, 6, 7) and new requests .

3.3 Code Implementation & Explanation

The program is implemented in Java and allows the user to select the default dataset provided in the project description or enter manual input. The core logic for each algorithm is implemented as follows:

1. First-In-First-Out (FIFO)

- **Logic:** FIFO replaces the page that has been in memory the longest, strictly following the order of arrival. It does not consider how often or how recently a page was accessed.
- **Implementation:** We utilized a `Queue` (LinkedList implementation) data structure. When a new page arrives and memory is full, the page at the head of the queue (the oldest) is removed, and the new page is added to the tail .
- The queue structure naturally models the arrival order of pages, which aligns directly with the FIFO replacement policy.

2. Least Recently Used (LRU)

- **Logic:** LRU replaces the page that has not been used for the longest period. It operates on the principle of "Locality of Reference," assuming that pages used recently are likely to be used again soon .
- **Implementation:** We used a `LinkedList` to track usage history:
    - **On Hit (Page exists):** The page is removed from its current position and re-added to the tail (marking it as "Most Recently Used").
    - **On Miss (Page Fault):** If memory is full, the page at the head of the list (which represents the "Least Recently Used") is removed.
- By updating page positions on every access, the data structure approximates recent usage history, which is the core assumption behind LRU.

3. Optimal Algorithm

- **Logic:** This algorithm replaces the page that will not be used for the longest period **in the future**. It serves as a theoretical benchmark for performance comparison, as it requires knowledge of future events.
- **Implementation:** The code uses a nested loop to "look ahead" in the reference string. When a replacement is needed, it calculates the distance to the next occurrence for every page currently in memory. The page with the greatest distance (or one that is never used again) is selected for eviction .
- The nested look-ahead mechanism simulates perfect future knowledge, allowing the algorithm to serve as a theoretical lower bound for page faults.

```java
import java.util.*;

/**
* PROJECT: Frame Replacement Algorithm (Task 2)
* * This program simulates the paging process in an operating system using
* three different page replacement algorithms:
* 1. First-In-First-Out (FIFO)
* 2. Least-Recently-Used (LRU)
* 3. Optimal Algorithm
* * It calculates and compares the number of "Page Faults" generated by each algorithm.
*/
public class framereplacement {

   public static void main(String[] args) {
       Scanner scanner = new Scanner(System.in);

       System.out.println("--- Frame Replacement Algorithm Simulation ---");

       // 1. Get the number of Frames (Physical Memory)
       // PDF Example suggests 10 frames
       System.out.print("Enter total number of frames (F): ");
       int frames = 0;
       try {
           frames = scanner.nextInt();
       } catch (Exception e) {
           System.out.println("Invalid input.");
           return;
       }

       // 2. Get the Reference String (Page Demands)
       // The user can input manually, or we can use the default sequence from the PDF.
       System.out.println("\nSelect input mode:");
       System.out.println("1. Enter page demands manually");
       System.out.println("2. Use the default 40-page sequence from
```

```
the Project PDF");
        System.out.print("Choice: ");
        int choice = scanner.nextInt();

        int[] referenceString;

        if (choice == 2) {
            // Load the specific sequence given in the assignment
table
            referenceString = getPDFExampleSequence();
            System.out.println("Loaded sequence: " +
Arrays.toString(referenceString));
            System.out.println("Total demands (N): " +
referenceString.length);
        } else {
            // Manual Input
            System.out.print("Enter total number of page demands (N):
");
            int n = scanner.nextInt();
            referenceString = new int[n];
            System.out.print("Enter the list of " + n + " page demands
(integers separated by space): ");
            for (int i = 0; i < n; i++) {
                referenceString[i] = scanner.nextInt();
            }
        }

        System.out.println("\n--- Simulation Results ---");

        // 3. Execute Algorithms
        int faultsFIFO = runFIFO(frames, referenceString);
        int faultsLRU = runLRU(frames, referenceString);
        int faultsOptimal = runOptimal(frames, referenceString);

        // 4. Comparison & Conclusion
        System.out.println("\n--- Comparison ---");
        System.out.println("FIFO Faults    : " + faultsFIFO);
        System.out.println("LRU Faults     : " + faultsLRU);
```

```
        System.out.println("Optimal Faults: " + faultsOptimal);

        int min = Math.min(faultsFIFO, Math.min(faultsLRU,
faultsOptimal));
        System.out.print("Conclusion: The most efficient algorithm(s)
for this scenario is/are: ");
        if (faultsFIFO == min) System.out.print("[FIFO] ");
        if (faultsLRU == min) System.out.print("[LRU] ");
        if (faultsOptimal == min) System.out.print("[Optimal] ");
        System.out.println();

        scanner.close();
    }

    /**
     * Helper method: Returns the specific page demand sequence from
the Project PDF.
     * Pages 3 (Tables 1 & 2).
     */
    private static int[] getPDFExampleSequence() {
        return new int[] {
            // Time 1-20: Sequential 1 to 20
            1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
            11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
            // Time 21-30
            1, 2, 1, 1, 2, 6, 7, 8, 9, 1,
            // Time 31-40
            5, 6, 7, 7, 11, 15, 16, 11, 20, 2
        };
    }

    /**
     * Algorithm 1: First-In-First-Out (FIFO)
     * Replaces the oldest page in memory (the one that arrived
first).
     * * @param capacity The number of memory frames available.
     * @param pages    The sequence of page requests.
     * @return The total number of page faults.
```

```
     */
    public static int runFIFO(int capacity, int[] pages) {
        // Use a Set to quickly check if a page exists in memory
        HashSet<Integer> memorySet = new HashSet<>(capacity);

        // Use a Queue to track the order of arrival (FIFO logic)
        Queue<Integer> fifoQueue = new LinkedList<>();

        int pageFaults = 0;

        for (int page : pages) {
            // Check if page is already in memory (Hit)
            if (memorySet.contains(page)) {
                // Do nothing for FIFO on a hit
                continue;
            } else {
                // Page Fault occurred
                pageFaults++;

                // If memory is full, remove the oldest page
                if (memorySet.size() == capacity) {
                    int oldest = fifoQueue.poll(); // Remove from head
of queue
                    memorySet.remove(oldest);      // Remove from set
                }

                // Add the new page
                memorySet.add(page);
                fifoQueue.add(page); // Add to tail of queue
            }
        }

        System.out.println("Algorithm: FIFO    | Total Page Faults: "
+ pageFaults);
        return pageFaults;
    }

    /**
```

```java
    * Algorithm 2: Least Recently Used (LRU)
    * Replaces the page that has not been used for the longest period
of time.
    *  capacity The number of memory frames available.
    *  pages    The sequence of page requests.
    * @return The total number of page faults.
    */
   public static int runLRU(int capacity, int[] pages) {
       // We use a LinkedList to store pages.
       // Logic: The end of the list is "Most Recently Used", the
front is "Least Recently Used".
       LinkedList<Integer> memoryList = new LinkedList<>();

       int pageFaults = 0;

       for (int page : pages) {
           // Check if page is already in memory
           if (memoryList.contains(page)) {
               // HIT: It is used again, so we move it to the end
(Most Recent)
               memoryList.remove((Integer) page); // Remove current
instance
               memoryList.add(page);              // Add to the end
           } else {
               // MISS: Page Fault
               pageFaults++;

               // If memory is full, remove the Least Recently Used
(Head of list)
               if (memoryList.size() == capacity) {
                   memoryList.removeFirst();
               }

               // Add the new page to the end (Most Recent)
               memoryList.add(page);
           }
       }
```

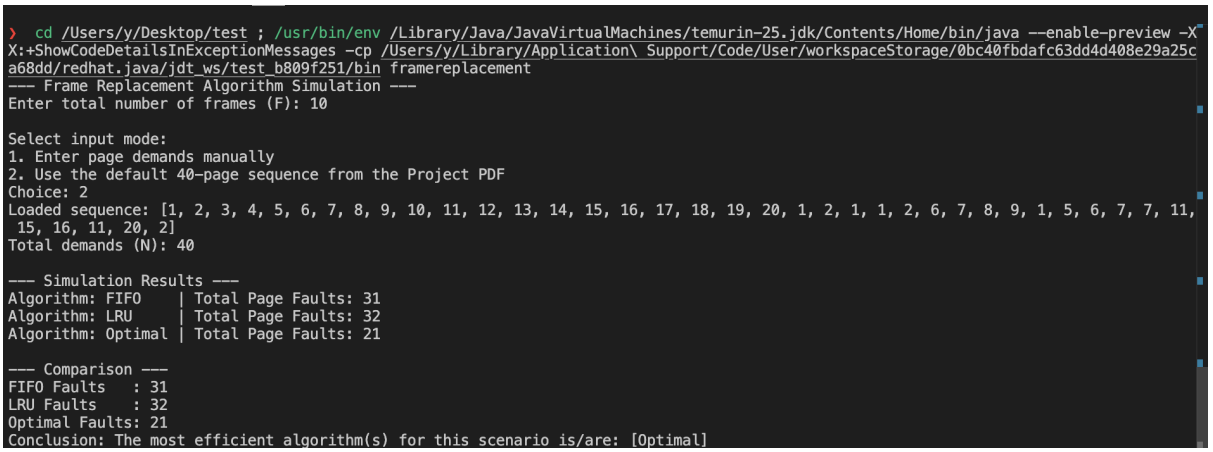
```
        System.out.println("Algorithm: LRU     | Total Page Faults: "
+ pageFaults);
        return pageFaults;
    }

    /**
     * Algorithm 3: Optimal Page Replacement
     * Replaces the page that will not be used for the longest period
of time in the FUTURE.
     * Note: This requires looking ahead in the reference string.
     * * @param capacity The number of memory frames available.
     * @param pages    The sequence of page requests.
     * @return The total number of page faults.
     */
    public static int runOptimal(int capacity, int[] pages) {
        // List to represent current frames
        ArrayList<Integer> memory = new ArrayList<>(capacity);

        int pageFaults = 0;

        for (int i = 0; i < pages.length; i++) {
            int currentPage = pages[i];

            // Check if page is already in memory
            if (memory.contains(currentPage)) {
                // Hit - no action needed for Optimal logic (future
doesn't change based on hit)
                continue;
            } else {
                // Page Fault
                pageFaults++;

                // If memory is not full, just add it
                if (memory.size() < capacity) {
                    memory.add(currentPage);
                } else {
                    // Memory is full, need to replace someone.
                    // We must find the page in memory that appears
```

```
FARTHEST in the future.

                    int indexToRemove = -1;
                    int farthestDistance = -1;

                    for (int j = 0; j < memory.size(); j++) {
                        int pageInMemory = memory.get(j);
                        int nextUseIndex = Integer.MAX_VALUE;

                        // Scan future requests starting from i+1
                        for (int k = i + 1; k < pages.length; k++) {
                            if (pages[k] == pageInMemory) {
                                nextUseIndex = k;
                                break; // Found the next usage
                            }
                        }

                        // If a page is never used again, it has the
max distance (optimal candidate)
                        if (nextUseIndex == Integer.MAX_VALUE) {
                            indexToRemove = j;
                            break;
                        }

                        // Otherwise, track who has the farthest next
use
                        if (nextUseIndex > farthestDistance) {
                            farthestDistance = nextUseIndex;
                            indexToRemove = j;
                        }
                    }

                    // Perform replacement
                    memory.set(indexToRemove, currentPage);
                }
            }
        }
```

```
        System.out.println("Algorithm: Optimal | Total Page Faults: "
+ pageFaults);
        return pageFaults;
    }
}
```

3.4 Output & Screen Capture

The following screenshot demonstrates the simulation output using the default 40-page sequence and 10 memory frames.



3.5 Discussion & Analysis

Based on the execution results shown in Section 3.4, the number of page faults generated by each algorithm is as follows :

- **FIFO:** 31 Page Faults
- **LRU:** 32 Page Faults
- **Optimal:** 21 Page Faults

Detailed Interpretation:

1. **The "Cold Start" Effect:**

   The reference string begins with 20 unique pages (1 to 20). Since the physical memory only has 10 frames, the first 10 requests fill the empty frames, and the subsequent 10 requests immediately trigger replacements. This results in **20 unavoidable page faults** regardless of the algorithm used . Therefore, the

algorithms effectively only compete on how they handle the remaining 20 requests (Steps 21-40).

2. **Performance of the Optimal Algorithm:**

   The Optimal algorithm achieved the theoretical minimum of 21 faults. This indicates that after the initial cold start, it only incurred 1 additional fault. By perfectly predicting future demand, it retained pages like 1, 2, 6, and 7, which were frequently accessed later, while evicting pages that would never be used again.This result confirms that the Optimal algorithm provides a theoretical lower bound on page faults for a given reference string and frame count.

3. **Comparison of FIFO vs. LRU:** In this specific scenario, FIFO (31 faults) performed slightly better than LRU (32 faults).

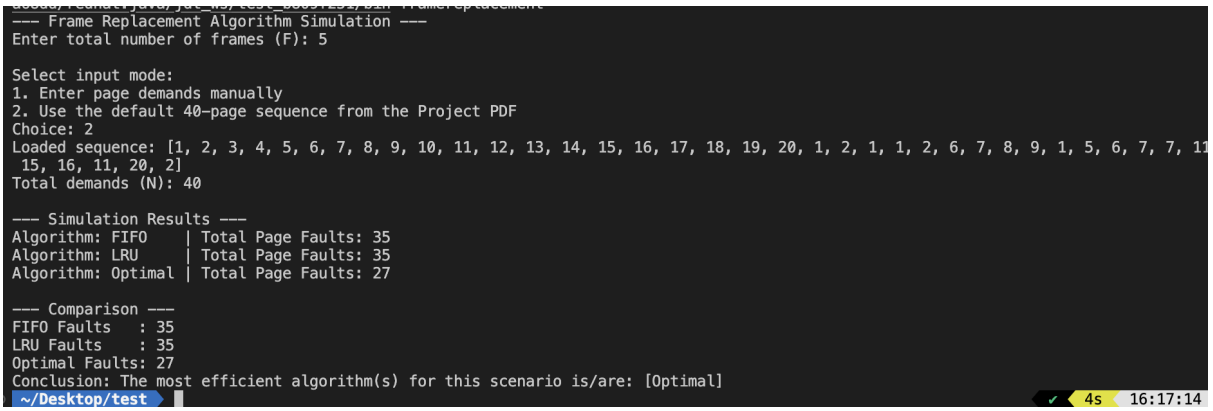
   Analysis: Although LRU is generally expected to outperform FIFO due to its reliance on temporal locality, this assumption does not always hold for every access pattern. it respects the "Locality of Reference," this specific dataset contains a pattern where "recent usage" did not perfectly predict "immediate future usage". LRU likely retained pages that were accessed recently but were not needed again immediately, occupying frames that could have been used for new arrivals. FIFO's simple eviction policy happened to align slightly better with the arrival pattern of this specific trace.This experiment illustrates how insufficient physical memory can degrade system performance regardless of replacement policy.

3.6 Extended Analysis (Sensitivity & Robustness)

To further evaluate the algorithms' stability, two additional experiments were conducted.

A. Impact of Reduced Frame Size (Sensitivity Analysis)

We re-ran the simulation using the same 40-page reference string but reduced the physical memory from **10 frames** to **5 frames**. This tests how the algorithms perform under high memory pressure.



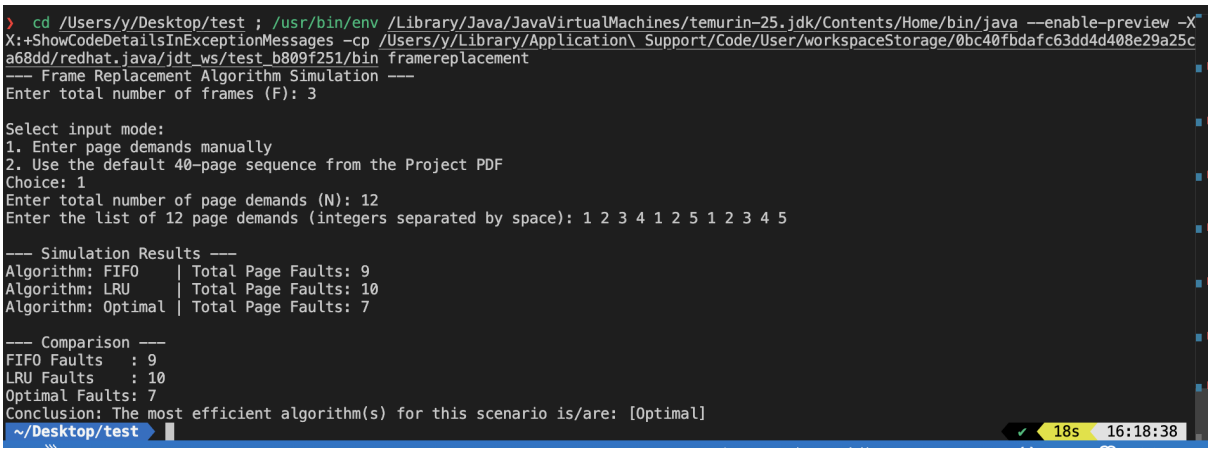
Observation:

- **Result:** The number of page faults increased significantly for all algorithms compared to the 10-frame scenario.
    - FIFO: **35**faults (increased from 31)
    - LRU: **35**faults (increased from 32)
    - Optimal: **27** faults (increased from 21)
- **Analysis:** With only 5 frames, the system cannot hold the initial set of unique pages, forcing frequent replacements. This simulates a condition known as "Thrashing," where the system spends more time swapping pages than executing tasks. The Optimal algorithm still maintained the lowest fault count, proving its efficiency is consistent regardless of memory size.This experiment illustrates how insufficient physical memory can degrade system performance regardless of replacement policy.

B. Manual Input Verification (Robustness Test)

To verify that the program correctly handles arbitrary user inputs, I manually tested a shorter sequence with a smaller frame count.

- **Settings:** 3 Frames.
- **Input Sequence (N=12):** `1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5`



Observation: The program successfully parsed the manual input and calculated the faults.

- **Result:**
    - FIFO: **9** Faults
    - LRU: **10** Faults
    - Optimal: **7** Faults
- **Analysis:** This test confirms that the implementation is flexible and not hard-coded to a single dataset. It correctly identified hits and misses for a dynamic user-provided string.

4.0 CONCLUSION

This project successfully simulated two critical components of operating system memory management: Memory Placement and Frame Replacement.

In **Part 1 (Memory Placement)**, the simulation results highlighted the trade-off between speed and space. The **First Fit** algorithm proved to be the fastest but caused uneven memory distribution by clustering processes at the start of the memory. The **Best Fit** algorithm demonstrated superior space management by utilizing smaller gaps (minimizing internal fragmentation) and preserving large contiguous blocks for future large processes. The **Next Fit** algorithm offered a balanced approach by distributing allocations more evenly across the memory partitions.

In **Part 2 (Frame Replacement)**, the **Optimal algorithm** outperformed all others by achieving the lowest possible number of page faults (21 faults). However, since it requires knowledge of future page requests, it serves only as a theoretical benchmark. Among the practical algorithms, **FIFO** slightly outperformed **LRU** (31 vs. 32 faults) in the 10-frame scenario, illustrating that LRU's assumption of locality does not hold for every access pattern. The sensitivity analysis further revealed that reducing physical memory significantly increases page faults, emphasizing the importance of adequate hardware resources to prevent thrashing.

Through this implementation, we gained a practical understanding of how operating systems balance efficiency, fairness, and resource utilization when managing limited memory resources.

4.1 Team Contribution

The project was collaboratively designed, implemented, and analyzed by the team. To ensure a comprehensive understanding of Operating System concepts, all members participated in the core logic design. The specific responsibilities were distributed as follows:

Member Name	Matrix No.	Roles & Responsibilities

YUE CHENGHAO	**227154**	**Project Lead & Core Developer** • Designed the OOP architecture (Partition Class) and main simulation framework. • Implemented the complex algorithms: **Best Fit** (Part 1) and **Optimal Algorithm** (Part 2). • Integrated all modules and handled code debugging/optimization. • Authored the core analysis sections (Introduction, Discussion, and Conclusion).
HUA JIE	226758	**Developer & Data Management** • Implemented standard algorithms: **First Fit / Next Fit** (Part 1) and **FIFO** (Part 2). • Developed the user input handling and validation logic (Scanner integration). • Assisted in verifying algorithm outputs against manual calculations.
WANG KAILUN	227046	**Testing & Documentation** • Conducted the **Sensitivity Analysis** (Section 3.6) by running simulations with reduced frames (5 and 3 frames). • Performed robustness testing (Test Case 3 - Error Handling).

		• Compiled the final report, managed formatting, and prepared screenshots.

5.0 REFERENCES

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*. 10th Edition. Wiley.
- [2] GeeksforGeeks. (2025). *First-Fit Allocation in Operating Systems*. Retrieved from https://www.geeksforgeeks.org/operating-systems/first-fit-allocation-in-operating-systems/
- [3] Tutorialspoint. (n.d.). *Operating System - Virtual Memory*. Retrieved from https://www.tutorialspoint.com/operating_system/os_virtual_memory.htm
- [4] Khanzada, A. (2023). *Contiguous Memory Allocation: First Fit, Best Fit, and Worst Fit*. Medium. Retrieved from https://medium.com/@khanzadaaneeda/contiguous-memory-allocation-first-fit-best-fit-and-worst-fit-734fd6f78ab
- [5] Mishra, M. (2024). *Page Replacement Algorithms - Exploring Operating Systems*. Retrieved from https://mohitmishra786.github.io/exploring-os/src/day-20-page-replacement-algorithms.html